



UNIVERSITÀ DI PISA

Dipartimento di Informatica
Corso di Laurea Triennale in Informatica

Corso a Libera Scelta - 6 CFU

Interazione Uomo-Macchina

Professori:

Prof. Daniele Mazzei
Tommaso Turchi
Roberto Figliè

Autore:

Filippo Ghirardini

Anno Accademico 2023/2024

Contents

1	Introduzione	4
1.1	Progettazione	4
1.1.1	Design	4
1.1.2	Pensiero computazionale	5
1.1.3	Confronto	5
2	Persone	6
2.1	Human Centered Design	6
2.1.1	Fasi di un processo	6
2.1.2	Activity Centered Design	6
2.2	Progettazione dell'interazione	7
2.2.1	Understanding	7
2.2.2	Discoverability	7
2.3	Perché delle azioni	9
2.3.1	Golfi	9
2.3.2	Stadi delle azioni	9
2.3.3	Fondamenti cognitivi	10
2.4	Personas	12
2.5	Requisiti	12
2.6	User stories	12
2.7	Scenari	13
2.7.1	Scrittura	13
2.8	Casi d'uso	13
2.8.1	Scrittura	14
2.9	Errore umano	14
2.9.1	Analisi della causa	14
2.9.2	Tipologie	15
2.9.3	Prevenzione	15
2.9.4	Mitigazione	16
2.10	User behavior patterns	16
2.11	Information architecture	17
2.11.1	Organization Schemes	17
2.11.2	Organizational Structures	18
2.11.3	Navigation	18
2.11.4	Componenti	18
2.12	Innovazione	19
2.12.1	Design Methods	19
2.12.2	Development methods	21
3	User Experience	22
3.1	Wireframing	22
3.2	Adaptability	22
3.2.1	Dispositivo	22
3.2.2	Utente	23
3.3	Wireflows	24
3.4	Mockups	24
3.5	Pretotyping	24
3.5.1	Mercato	24
3.5.2	Pretotype	25
3.6	Prototyping	25
3.6.1	Fidelity	26
3.6.2	Scope	26
3.6.3	Tecniche	27
3.7	Design systems	27
3.7.1	Atomic Design	28

3.7.2	Design Tokens	28
3.8	Evaluation	28
3.8.1	Definizioni	28
3.8.2	Valutazioni euristiche	29
3.8.3	Cognitive Walkthrough	30
3.8.4	Usability testing	31
3.9	IOT	33
3.9.1	Il Ciclo di Vita dell'Esperienza	33
3.9.2	Livelli di Design e Integrazione Aziendale	33
3.9.3	User Experience	34
3.10	Artificial Intelligence	34
3.10.1	Differenze	34
3.10.2	Affidamento	34
3.10.3	Miglioramento	35
3.10.4	Design	35

Interazione Uomo-Macchina

Realizzato da: Ghirardini Filippo

A.A. 2025-2026

1 Introduzione

L'obiettivo del corso è di studiare gli strumenti necessari a comprendere e gestire il processo di sviluppo delle interfacce e dei prodotti interattivi. È fondamentale che l'informatico crei un prodotto **apprezzato** dalle persone.

1.1 Progettazione

1.1.1 Design

Definizione 1.1.1 (Design). *Per design si intende sia il processo di **progettazione** e **pianificazione** sia l'**output** stesso di questo processo.*

Nel design il primo passo è capire **perché** un problema esiste e se ha davvero bisogno di una soluzione. È quindi fondamentale fare in modo che il software risolva i problemi dell'utente, senza aggiungere funzionalità inutili non necessarie all'utente.

Nel design del prodotto è spesso necessario modificare requirement e specifiche per andare in contro alle esigenze degli utenti, rendendo quindi lo sviluppo un processo **antropocentrico** di adattamento del software.

Esistono varie sotto discipline del design, tra cui:

- **Interaction design**: l'attività di progettazione dell'interazione che avviene tra esseri umani e oggetti in generale. Ha come obiettivo quello di rendere macchine, servizi e sistemi **usabili** per gli utenti target, mettendo al centro i loro **bisogni**. Si suddivide in:

- Design di **prodotto**: progettazione di beni e servizi il cui obiettivo principale è quello di essere utilizzati da quanti più utenti possibili migliorandone la vita¹

Definizione 1.1.2 (Designer). *L'inventore o designer del prodotto si focalizza su un problema da risolvere e inventa un prodotto che grazie alle sue caratteristiche tecniche permette di risolvere un problema.*

- Design dell'**esperienza utente** o **UX**: il processo volto ad aumentare la soddisfazione e la fedeltà del cliente migliorando l'**usabilità** e la piacevolezza. Aggiunge alla tradizionale progettazione gli aspetti di business, marketing e sviluppo prodotto.

Definizione 1.1.3 (UX Designer). *Lo UX Designer ha l'obiettivo di far vivere all'utente del suo prodotto la miglior esperienza possibile, evitando di farlo sentire frustrato e deluso.*

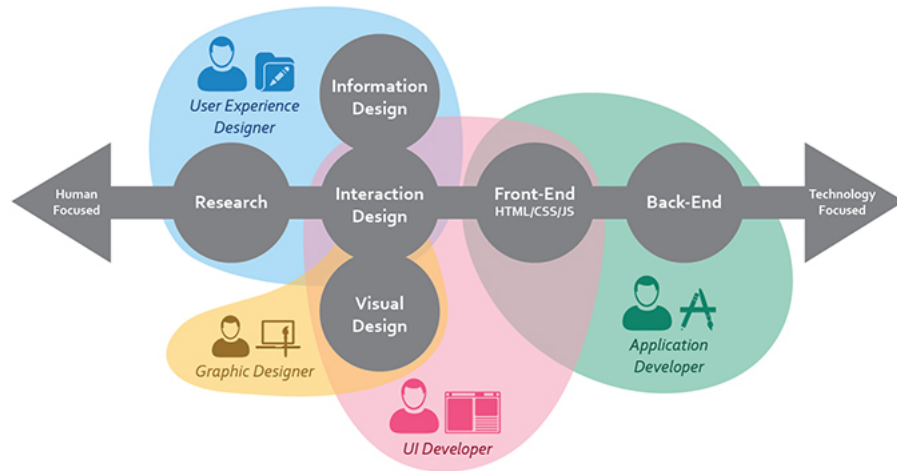
- Design dell'**interfaccia** o **UI**: ha come focus il modo in cui le persone interagiscono con la tecnologia e cerca di migliorare la loro comprensione di ciò che si può fare. Dalla UX si crea uno schema di interazione che serve a progettare l'interfaccia utente al fine di abilitare l'esperienza progettata.

Definizione 1.1.4 (UI Designer). *Lo UI Designer progetta l'aspetto **estetico** e la struttura dell'interfaccia così che questa durante l'utilizzo induca l'utente nel seguire l'esperienza progettata. Produce un **wireframe**² dell'interfaccia e delle linee guida che verranno seguite dagli **UI Developer** per l'implementazione.*

- **Human Machine e Human Computer Interaction**: ha come obiettivo quello di rendere possibile e **facilitare** al massimo per un essere umano l'**uso** e l'**interazione** con i sistemi del mondo IT

¹Spesso usato in modo interscambiabile con **design industriale**: processo strategico di risoluzione dei problemi che guida l'innovazione e porta a una migliore qualità della vita attraverso prodotti, sistemi, servizi ed esperienze innovative

²Bozza grafica



1.1.2 Pensiero computazionale

Nel mondo del *design*, in particolare quello industriale, ci si approccia alla progettazione e risoluzione dei problemi in modo diverso dal mondo informatico.

Per questo motivo nel 2006 Jeannette Wing, direttrice del dipartimento di Informatica della Mellon University, formula la seguente definizione:

Definizione 1.1.5 (Pensiero computazionale). *Il pensiero computazionale è un processo di formulazione di problemi e soluzioni in una forma che sia eseguibile da un agente che processi informazioni.*

Questo consiste non solo nel saper programmare ma anche nel pensare a diversi **livelli di astrazione**, abituando al *rigore* e rendendo possibili gli *atti creativi*. Permette di interagire con persone e strumenti e di usare le macchine come oggetti capaci di compensare le lentezze o l'imprecisione dell'uomo.

1.1.3 Confronto

Se nel **design** la progettazione è affrontata in maniera globale, nel tentativo di comprendere il problema nel suo insieme, nel **pensiero computazionale** si basa sulla suddivisione di un problema in sotto-problemi in modo da giungere ad una formulazione del problema come *algoritmo*.

È importante evidenziare che il design dell'interazione e il pensiero computazionale non sono mutualmente esclusivi ma anzi è nell'**unione** dei due e nella loro **integrazione** che si crea software di qualità.

2 Persone

2.1 Human Centered Design

I progettisti devono produrre cose che soddisfino i bisogni della gente in termini di funzioni, facilità d'uso e gratificazione emotiva, ovvero come un'esperienza totale.

L'obiettivo è quello di contrastare la sempre maggiore frustrazione dovuta alla complessità degli oggetti quotidiani: se le macchine hanno una modalità di funzionamento *logica*, gli umani sono l'opposto e di conseguenza nell'interazione tra i due si va a creare una relazione fra specie diverse con modalità di pensiero e funzionamento opposte.

Definizione 2.1.1 (Human Centered Design). *È una **forma di pensiero** per lo sviluppo dei sistemi interattivi che mira a renderli **utilizzabili** e **utili** concentrandosi sugli utenti e i loro bisogni. Questo approccio migliora l'**efficacia**, l'**efficienza**, la **soddisfazione dell'utente**, l'**accessibilità** ed evitando effetti negativi sulla salute, sicurezza e performance.*

Nel design **antropocentrico** si inverte il paradigma di progettazione (*tecno-centrico*) e si mette l'utente e i suoi bisogno al centro del processo, invece delle funzionalità del prodotto. Per farlo è fondamentale **capire le persone** osservandole, dato che spesso loro stessi non sanno di cosa hanno bisogno.

È soprattutto fondamentale la **comunicazione**, specialmente dalla macchina all'utente e in particolare quando qualcosa va storto in modo che l'utente venga guidato nella risoluzione e gestione dell'errore.

L'obiettivo deve essere quello di creare **empatia** nell'utente. I ricordi hanno la capacità di far provare emozioni più profonde rispetto al presente e un utente che ha memoria di un successo favorito dalla tecnologia svilupperà empatia e bisogno per quel prodotto.

Esempio 2.1.1 (Bitcoin). La tecnologia *blockchain* è nata in quanto c'era un bisogno per un metodo alternativo di scambiare moneta (cryptomoneta). In seguito si è provato ad applicare questa nuova tecnologia ad altri ambiti ma senza successo in quanto non si stava creando una soluzione ad un problema ma si stava cercando un problema per una soluzione.

2.1.1 Fasi di un processo

Un processo HCD può essere schematizzato come un flusso continuo ed iterativo che attraversa le seguenti fasi:

- Specificare il **contesto d'uso**: identificare la user base, per cosa utilizzeranno il prodotto e sotto quali condizioni e vincoli
- Specificare i **requirements**: identificare i business requirements e gli obiettivi utente che devono essere raggiunti utilizzando il software
- **Progettare la soluzione**: può essere divisa in più sotto fasi iterative, tipicamente dalle *bozze* ai *prototipi* e alla *soluzione*
- **Testare e valutare**: fondamentale per iniziare di nuovo il ciclo in base ai risultati e procedere quindi ad un miglioramento incrementale

Note 2.1.1. È fondamentale implementare nel software sistemi di **tracking** dell'utente finalizzati alla produzione di statistiche di utilizzo.

2.1.2 Activity Centered Design

È un'estensione di HCD e consiste nel mettere enfasi sulle attività che un utente potrebbe fare con una determinata tecnologia.

I **controlli** Activity Centered (e.g. interruttore per accendere il proiettore) sono eccellenti in teoria ma se realizzati male crea molte difficoltà. Devono quindi essere **User-Activity-Centered** e non Device-Centered

2.2 Progettazione dell'interazione

Il problema principale è che il *buon design* non è universale in quanto un prodotto non può essere apprezzato da tutti perché l'esperienza è soggettiva e dipende dalla persona.

Le macchine sono limitate a seguire **regole fisse e rigide** e si comportano come programmate anche se l'utente si discosta leggermente da esse, nonostante questo possa significare fare qualcosa di totalmente insensato. Questo perché, a differenza degli umani, non hanno il **buon senso** e inoltre spesso le regole da esse seguite sono conosciute solo dalla macchina e dai designers.

Sviluppatori ed ingegneri tendono a compiere l'errore di pensare che la spiegazione logica sia sufficiente per avere un buon design, mentre ovviamente non è così.

Esempio 2.2.1 (Three Mile Island accident). L'incidente al reattore nucleare fu causato da un pessimo sviluppo dell'interfaccia di controllo: nonostante una valvola fosse bloccata in posizione aperta, il led che doveva indicare se fosse chiusa era acceso. Gli operatori impiegarono diverse ore prima di scoprire che indicava solo se il solenoide riceveva corrente e non lo stato di chiusura.

2.2.1 Understanding

Definizione 2.2.1 (Understanding). *È la capacità del prodotto di farsi usare correttamente dall'utente.*

In pratica è la proprietà associata a quanto bene un prodotto dice come si usano le sue funzioni disponibili. È necessario che risponda alle seguenti domande:

- **Come** si usa il prodotto?
- Che **funzione** ha ciascun controllo?
- Come si **combinano** i controlli?

2.2.2 Discoverability

Definizione 2.2.2 (Discoverability). *È la capacità di un sistema di veicolare e comunicare i propri possibili usi all'utente.*

L'idea è quindi di far capire all'utente a cosa serve il prodotto, tipicamente lavorando sulla **visibilità**. Questo non implica che poi lo si sappia usare.

Questa caratteristica si basa su sei principi fondamentali:

- **Affordances**: il termine si riferisce alla **relazione** tra le *proprietà di un oggetto* fisico e le *capacità di un agente* che determina come un oggetto possa essere utilizzato. Il contrario è **anti-affordance**, ovvero la prevenzione dell'interazione.

Esempio 2.2.2. La maggior parte delle sedie possono essere trasportate da una persona (they *afford* lifting). Se una persona debole non riesce a sollevarla, it *doesn't afford* lifting.

Perché questo principio sia efficace è necessario che le affordances e le anti-affordances siano **discoverable** e **perceivable**. Un esempio negativo è il vetro.

- **Signifiers**: specifica **dove** l'azione deve essere eseguita (non **quale**) e comunicano quindi come usare il design. Possono essere **deliberati** ed **intenzionali**, come il segno "spingi" sulla porta, o **accidentali** e **non intenzionali**, come un percorso nell'erba causato da tante persone che ci hanno camminato.

Per passare da un'affordance a un signifier di solito si procede tramite **convenzioni**, come ad esempio il fatto che una maniglia su una porta indica che devi usarla per aprirla. L'interpretazione di una *perceived affordance* è una **convenzione culturale**.

- **Mapping:** indica la **relazione** tra gli elementi di due insiemi di cose. È importante soprattutto nel design e layout dei controlli e degli schermi in quanto utilizza **analogie spaziali** tra i controlli e i dispositivi controllati per indicare come utilizzarli (e.g. fornelli). Alcuni mappings sono biologici (muovere la mano in alto e in basso indicano più e meno) mentre altri culturali (indicatore del carburante pieno a destra o sinistra).
- **Feedback:** la comunicazione del risultato di un'azione. Deve essere **immediato** ed **informativo** senza ostruire altro (e.g. troppe notifiche ti portano a disattivarle completamente).
- **Conceptual model:** una spiegazione, di solito molto semplificata, di come un qualcosa funziona (e.g. cartelle di file sul pc). Non deve essere completa o accurata, basta che sia utile. È fondamentale che l'**assunzione** su cui si basa la semplificazione rimanga vera (e.g. far vedere i file anche se sono nel cloud, la connessione deve esserci). Sono spesso inferiti dal dispositivo stesso, tramandati da persone, estratti da manuali o costruiti con l'esperienza.
I **mental model** sono invece come le persone rappresentano il funzionamento delle cose nella loro mente. È diverso per ognuno e una persona singola può averne anche diversi per lo stesso oggetto, a volte anche in conflitto.
La combinazione di informazioni necessarie per la creazione di un *conceptual model* di un dispositivo con cui interagiamo è la **system image**. Ad esempio aspetto fisico, somiglianza con altri dispositivi già usati, pubblicità, articoli di giornale o manuali.
Quando la system image è incoerente o inappropriata, l'utente ha difficoltà nell'usare il dispositivo. È importante che il designer trasmetta il suo conceptual model correttamente, tramite la system image, al conceptual model dell'utente.
- **Constraints:** servono a limitare l'insieme delle possibili azioni grazie alla conoscenza del mondo e i modelli concettuali. Possono essere
 - **Fisici**, tra cui le **forcing functions** ovvero casi estremi di constraints che evitano comportamenti inappropriati. Si suddividono in:
 - * **Interlocks:** impone che una serie di operazioni siano eseguite nel giusto ordine
 - * **Lock-ins:** mantiene un'operazione attiva evitando che qualcuno la interrompa prematuramente (e.g. conferma per la chiusura di un programma)
 - * **Lockout:** evita che qualcuno acceda ad uno spazio pericoloso o evita che accada un determinato evento (e.g. limite di età)
 - **Culturali**
 - **Semantici**
 - **Logici**

Note 2.2.1 (Consistenza). Dato che violare delle convenzioni richiede nuovo apprendimento, mantenere una **consistenza** nei vari design è una caratteristica importante.

2.3 Perché delle azioni

È importante analizzare come le persone **sceglono** e **valutano** le loro azioni per costruire un modello concettuale della psicologia umana. Questo implica capire le **emozioni**: **piacere** e **frustrazione** in base al successo delle azioni.

2.3.1 Golfi

Quando le persone usano qualcosa, affrontano due golfi. È compito del *designer* aiutarle a connetterli.

- **Esecuzione**: sforzo mentale per tradurre gli obiettivi in azioni fisiche sul sistema. È correlato alla *discoverability* (l'utente capisce cosa è possibile fare?), alle *affordances* (l'utente interpreta cosa un componente permette di fare?) e alla capacità fisica di eseguire l'azione. I principali elementi del design coinvolti nella connessione sono:
 - *Signifiers*
 - *Constraints*
 - *Mappings*
 - *Conceptual model*
- **Valutazione**: capire se i cambiamenti percepiti hanno avvicinato il sistema all'obiettivo. Richiede un *feedback* continuo sui risultati e sullo stato corrente: l'utente deve sviluppare un senso di *controllo* e comprensione. È importante la **percezione** dell'utente sulle informazioni visuali. I principali elementi del design coinvolti nella connessione sono:
 - *Feedback*
 - *Conceptual model*

2.3.2 Stadi delle azioni

Un'azione si compone dell'**esecuzione** e dell'**interpretazione** dei risultati. Entrambi hanno un effetto sullo stato emotivo dell'utente. Più precisamente dividiamo l'azione in sette stadi:

1. Creazione dell'**obiettivo**
2. **Pianificazione** dell'azione
3. **Specificazione** di un'azione
4. **Esecuzione** dell'azione
5. **Percezione** dello stato del sistema
6. **Interpretazione** della percezione
7. **Confronto** del risultato con l'obiettivo

Le azioni sono **non lineari**: non è necessario attraversare tutti gli stadi, spesso servono più azioni per un singolo obiettivo, possono formarsi sotto-obiettivi e sotto-piani oppure determinati obiettivi possono essere dimenticati, scartati o modificati.

I sette stadi forniscono una linea guida per lo sviluppo di prodotti o servizi; possono essere visti come sette **domande** alle quali chiunque crei un prodotto dovrebbe sempre saper rispondere:

1. Cosa voglio ottenere?
2. Quali sono azioni alternative?
3. Che azione posso fare ora?
4. Come la faccio? Grazie al **feedforward**, che sfrutta *signifiers*, *constraints* e *mappings*
5. Cosa è successo? Grazie al **feedback**, che utilizza le informazioni esplicite sull'impatto dell'azione
6. Cosa significa?
7. Ho raggiunto il mio obiettivo?

Grazie ai sette stadi dell'azione possiamo definire il sette **principi fondamentali del design**:

1. **Discoverability**: è possibile determinare quali azioni sono possibili e lo stato attuale del dispositivo
2. **Feedback**: c'è un flusso continuo e completo di informazioni sul risultato delle azioni e sullo stato del prodotto
3. **Conceptual model**: il design mostra tutte le informazioni necessarie per la creazione di un conceptual model del sistema che permetta la *comprensione* e il *sensu di controllo*
4. **Affordances**: esistono le affordances appropriate per fare le azioni desiderate
5. **Signifiers**: uso efficace dei signifiers per garantire *discoverability* e un *feedback* funzionale e comprensibile
6. **Mappings**: la relazione tra i controlli e le azioni segue il principio di mapping, migliorato quanto possibile attraverso il *layout spaziale* e la *continuità temporale*
7. **Constraints**: fornire constraints di tutti i tipi guida le azioni e facilita l'interpretazione

2.3.3 Fondamenti cognitivi

Innanzitutto possiamo dividere le azioni in due tipi:

- **Opportunistiche**: meno precise ma richiedono meno sforzo mentale, meno problemi e probabilmente più interesse
- **Pianificate**

Inoltre, il pensiero umano si divide ulteriormente in:

- **Conscio**: lento ed elaborato, si analizzano le decisioni, si pensa ad alternative e si valutano diverse opzioni con logica e matematica. Utilizzato durante l'apprendimento, in situazioni di pericolo o quando qualcosa va storto
- **Subconscio**: il cervello confronta la situazione attuale con quelle passate e agisce velocemente e senza sforzo. È ottimo nel riconoscimento di trend e in generale in comportamenti "skilled"

Questo ci porta a definire i due tipi di **memoria**:

- **Dichiarativa**, e.g. ricordare la capitale di un dato paese
- **Procedurale**, e.g. ricordando le attività fatte per arrivare a compiere quell'azione in passato

La cognizione umana è ovviamente **limitata**, sia per *attenzione*, sia *memoria* e che per *ragionamento*. Gli utenti sfruttano aiuti esterni come note o indizi nell'interfaccia: questo implica per il design che il sistema deve compensare per queste limitazioni.

Livelli di elaborazione Un'approssimazione semplicistica di come il cervello elabora le informazioni prevede tre livelli:

- **Viscerale**: comprende i meccanismi di protezione che permettono di fare valutazioni veloci ma semplici sull'ambiente, permettendo di rispondere velocemente e in maniera subconscia. Per i designer questo riguarda la *percezione immediata* (e.g. suono piacevole o meno) e nulla c'entra con l'effettivo funzionamento del prodotto.
- **Comportamentale**: riguarda le abilità apprese che vengono attivate quando si presenta la giusta situazione. Principalmente a livello *subconscio* ma comunque la persona rimane cosciente delle azioni (ma non dei dettagli). Per i designer questo implica che ad ogni azione è associata un'**aspettativa**, positiva o negativa, che porta a *sensu di controllo* o *frustrazione*. È fondamentale il *feedback* per la gestione di questo aspetto.

- **Riflessivo:** riguarda il pensiero *conscio* ed è un processo lento e profondo che avviene dopo che l'evento è successo e comprende le emozioni più complesse. Per i designer è la parte più importante in quanto è qui che si crea un'opinione del prodotto e una memoria di esso.

Riassumendo:

Livello Cognitivo	Fasi Principali	Implicazioni di Design
Viscerale	4. Eseguire 5. Percepire	• Le prime impressioni contano • Attrazione visiva ed estetica • Feedback sensoriale immediato • "Sembra/si sente bene?"
Comportamentale	3. Specificare 6. Interpretare	• Interazioni fluide e abituali • Significanti e mappature chiari • Pattern coerenti • "Posso farlo automaticamente?"
Riflessivo	1. Obiettivo 2. Pianificare 7. Confrontare	• Comprensione più profonda • Modelli concettuali • Memoria e soddisfazione • "Perché è successo? Lo consiglierai?"

Esempio 2.3.1 (Cestino). La metafora fisica del cestino reale con quello digitale. Il **modello mentale** dell'utente è che i file eliminati vanno nel cestino e non sono permanentemente persi finché non si svuota. Funziona perché:

- Riduce il **golfo dell'esecuzione**: gli utenti sanno come eliminare, trascinando i file sul cestino
- Riduce il **golfo di valutazione**: c'è un feedback visivo, l'icona del cestino si riempie e i file sono visibili al suo interno
- Corrisponde all'esperienza nel mondo reale
- Fornisce sicurezza in quanto azione reversibile

Esempio 2.3.2 (Errori senza contesto). Il classico errore di Windows di una stampa fallita è un pessimo esempio in quanto:

- Aumenta il **golfo dell'esecuzione**: l'utente non sa cosa fare dopo, può solo cliccare "OK" per ignorarlo
- Aumenta il **golfo di valutazione**: il messaggio di errore è vago e non specifica quale sia il vero problema se non con un codice di errore che per l'utente non ha significato
- "Nascondi dettagli" suggerisce che ci siano più informazioni, ma in realtà è solo nascosto di default
- Non si crea un **modello concettuale** in quanto l'utente non capisce che cosa sia andato storto

2.4 Personas

La *user persona* è il personaggio che rappresenta un potenziale utente del prodotto e servono ad aiutare il team di design nel loro lavoro, analizzando e dividendo gli utenti per i loro **obiettivi** e le loro **frustrazioni**. La creazione di una *persona* parte dalle analisi degli utenti tramite tecniche, come:

- **Analisi dei task**, e.g. ordinamento di carte
- **Feedback**, e.g. interviste e focus groups
item **Prototyping**, ovvero sperimentare con le idee prima di svilupparle

Questo porta alla definizione di tre tipi di *personas*:

- **Proto**: una versione leggera creata apposta in poco tempo dai designers
- **Qualitative**: create da ricerche esplorative solide (e.g. interviste) con un campione da cinque a trenta utenti, poi segmentati
- **Statistical**: raccolta di dati tramite un sondaggio su un grande numero di utenti per poi cercare *cluster* di risposte analoghe. In realtà funzionano in combinazione con la *qualitative research*

Note 2.4.1 (Pareto principle). Quante persone considerare per la ricerca? Per tanti eventi, circa l'80% degli effetti deriva dal 20% delle cause.

All'interno di ogni *persona* troviamo diverse informazioni:

- Informazioni **demografiche**, e.g. età e salario
- **Dettagli personali**, e.g. biografia e fotografie
- **Comportamento** e/o dettagli cognitivi, come il modello mentale e come si sente riguardo alle attività da compiere
- **Obiettivi e motivazioni** per l'utilizzo del prodotto
- Dettagli **comportamentali** su come la persona si comporta durante l'utilizzo del prodotto

Note 2.4.2 (Archetype). La *persona* è quella fisica mentre l'astrazione è l'**archetipo**.

2.5 Requisiti

Definizione 2.5.1 (Requisito). *Un requisito è un servizio, funzione o caratteristica di cui un utente ha bisogno.*

È importante specificare che i requisiti sono dinamici tanto quanto l'ambiente in cui nascono. Possiamo comunque dividerli in due categorie:

- **Funzionali**: esprimono una funzione o caratteristiche e definiscono cosa è necessario, e.g. visitare il sito del cliente. **Non** includono **come** implementare fisicamente la soluzione.
- **Non funzionali**: definiscono quanto bene o a che livello una soluzione si trova, tramite caratteristiche come sicurezza, manutenibilità, disponibilità, etc. . . Possono essere relativi ad un gruppo di *requisiti funzionali* o ad un singolo

2.6 User stories

Definizione 2.6.1 (User story). *È una breve frase o riassunto che identifica l'utente e il loro obiettivo/necessità dal loro punto di vista.*

Le *user stories* aiutano a descrivere le informazioni pratiche sugli utenti, e.g. necessità e motivazioni per un servizio, e aiutano il team di sviluppo a creare una *roadmap*.

As a [role], I want [feature] because [reason].

Possono essere scritte da chiunque, ma è necessario precisare che non serve che sia il proprietario del prodotto a farlo. In un progetto con sviluppo agile, ogni componente del gruppo dovrebbe scriverle. In ogni caso è più importante il protagonista della *story* rispetto all'autore. Possono essere scritte in diversi livelli di dettaglio, anche se tendenzialmente quelle più grandi vengono divise in molteplici piccole.

2.7 Scenari

Definizione 2.7.1 (Scenario). *Uno scenario è una situazione che descrive come gli utenti eseguono azioni sul tuo sito o applicazione.*

Gli scenari sono in pratica lo sviluppo della **user story** aggiungendo l'**interazione** con il prodotto o il servizio. Possono essere divisi in più casi d'uso.

Esempio 2.7.1. Uno scenario in cui John usa un'applicazione per il telefono per comprare un biglietto per un workshop di design mentre va a lavoro.

Gli scenari aiutano gli stakeholders a visualizzare l'idea del team di design, fornendo un contesto per come dovrebbe essere l'esperienza dell'utente, permettendo la comunicazione tra il pensiero *creativo* e quello di *business*. Aiutano anche il team di design permettendogli di immaginare la soluzione ideale per il problema di un utente e su quali parti della UX concentrarsi.

2.7.1 Scrittura

Un buon scenario deve essere **conciso** e rispondere alle seguenti domande:

- **Chi** è l'utente? Usando le *personas* sviluppate
- **Perché** l'utente usa il prodotto? Quali sono le sue motivazioni e aspettative?
- Quali **obiettivi** ha? Tramite l'*analisi dei task* capire come soddisfarli
- **Come** fa l'utente a raggiungere gli obiettivi? Quali ostacoli e possibilità ci sono? Non è sempre presente

La pianificazione parte con il **mapping**: l'utente principale viene definito tramite sviluppo di *persona* e l'**attività chiave** è quella che l'utente spera di completare. Il passo successivo è fare un'*analisi* dello scenario, mettendo gli obiettivi dell'utente nel contesto e vedere quali passi dovrebbe fare.

Note 2.7.1. È fondamentale concentrarsi sugli obiettivi dell'utente: ciò che vogliono ottenere dal servizio, il loro contesto, la loro conoscenza pregressa e il loro background.

2.8 Casi d'uso

Un caso d'uso è una **descrizione scritta** di come un utente eseguirà l'azione nell'applicazione e specifica, dal suo punto di vista, il comportamento del sistema. Ogni caso d'uso è rappresentato come una sequenza di passi a partire dall'obiettivo e fino al suo raggiungimento.

La differenza principale dallo *scenario* è la **prospettiva**: il caso d'uso è più granulare e descrive il **flusso** di uno specifico attore. Il caso d'uso aggiunge valore spiegando **come** il sistema si dovrebbe comportare e aiutando a capire cosa potrebbe andare storto. Fornisce una lista di obiettivi che può essere utilizzata per stabilire il costo e la complessità del sistema.

Incluso	NON incluso
• Chi sta usando il sito web • Cosa vuole fare l'utente • L'obiettivo dell'utente • I passaggi che l'utente compie per completare un compito specifico • Come il sito web dovrebbe rispondere a un'azione	• Linguaggio specifico per l'implementazione • Dettagli sulle interfacce utente o sulle schermate

In particolare un caso d'uso si compone dei seguenti **elementi**:

- **Attore**: qualcuno o qualcosa che usa il sistema con un certo comportamento
- **Stakeholder**: qualcuno o qualcosa che ha interesse nel comportamento del sistema in discussione (SUD)
- **Attore principia**: stakeholder che inizia un'interazione con il sistema per raggiungere un obiettivo
- **Precondizioni**: cosa deve essere vero prima e dopo l'esecuzione del caso d'uso
- **Triggers**: evento che causa il caso d'uso
- **Basic flow**: caso d'uso dove va tutto bene
- **Alternative flow**: percorsi alternativi che accadono quando qualcosa va storto nel sistema

2.8.1 Scrittura

Kenworthy (1997) specifica i seguenti passaggi per scrivere un caso d'uso:

1. Identificare chi userà il sito web
2. Scegliere uno di quegli utenti
3. Definire cosa l'utente vuole fare sul sito. Ogni azione fatta diventa un caso d'uso
4. Per ogni caso d'uso, decidere cosa dovrebbe normalmente succedere
5. Descrivere il corso degli eventi in termini di cosa l'utente fa e come il sistema si comporta nei suoi confronti
6. Considerare corsi degli eventi alternativi
7. Trovare similarità tra i vari casi d'uso, estrarle e usarle per creare un caso d'uso comune
8. Ripetere da 2 a 7 per tutti gli altri utenti

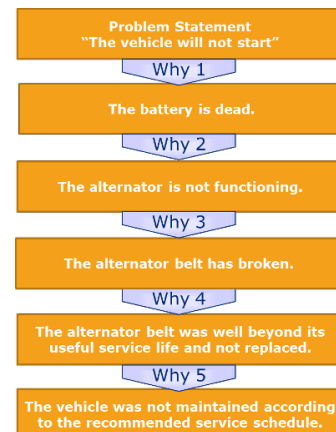
2.9 Errore umano

La maggior parte degli incidenti industriali sono causati da errori umani (c.a. 75% – 95%) e la colpa è del design. Questo perché progettiamo macchinari che richiedono attenzione e allerta costante per ore o procedure arcaiche e confusionarie che vengono usate una volta nella vita. Le persone si ritrovano quindi senza nulla da fare per ore con all'improvviso una situazione in cui bisogna rispondere velocemente e precisamente oppure situazioni in cui sono costantemente **interrotte** mentre devono compiere più attività contemporaneamente.

In generale l'essere umano non è bravo nella precisione: eccelle in creatività, curiosità ed esplorazione. Tutte cose che vanno contro il funzionamento delle macchine.

2.9.1 Analisi della causa

Quando si presenta un errore è fondamentale capire **perché** sia successo. Una buona tecnica è quella sviluppata da Sakichi Toyoda, i "Five Whys", ovvero continuare a chiedere per cinque volte perché partendo dal problema iniziale. In pratica non è detto che ne servano o bastino cinque.



2.9.2 Tipologie

Definizione 2.9.1 (Errori). *L'errore umano è definito come un qualunque discostamento dal comportamento "appropriato" (che spesso è determinato a fatto avvenuto). È il termine generale per tutte le azioni sbagliate.*

Esistono due classi principali di errori:

- **Slips** o lapsus: avviene quando una persona vorrebbe fare un'azione ma finisce per farne un'altra. Di solito fatti dai *neofiti*. Si divide in:
 - **Action-based**: l'azione sbagliata viene eseguita, e.g. verso il latte nel caffè e poi metto l'ultimo in frigo invece del primo
 - **Memory lapses**: la memoria fallisce e l'azione non viene eseguita o risulta non valutata, e.g. dimentico di spegnere il fornello
- **Mistakes** o errori cognitivi: avviene quando viene stabilito l'obiettivo sbagliato o viene formato il piano sbagliato. Da quel momento, anche se le azioni sono eseguite correttamente, fanno tutte parte dell'errore. È quindi il **piano** ad essere sbagliato. Di solito fatti dagli *esperti*. Si divide in:
 - **Rule-based**: la persona ha capito la situazione ma poi decide in base ad un corso di eventi errato, viene seguita la regola sbagliata. E.g. un meccanico trova un problema alla batteria ma decide di non rimpiazzarla perché funziona alla metà delle prestazioni
 - **Knowledge-based**: il problema non viene diagnosticato per conoscenza errata o incompleta, e.g. il peso del carburante è stato calcolato in libbre invece che chili
 - **Memory-lapse**: c'è una dimenticanza nelle fasi di definizione degli obiettivi, piani o valutazione. E.g. un meccanico non trova il problema perché dimentica un passaggio durante un controllo

Interruzioni Le interruzioni sono una delle cause principali degli errori, specialmente i *memory-lapse*. Quando un'attività è interrotta da un altro evento, non solo c'è il tempo sprecato per risolverne la causa ma c'è anche quello necessario a riprendere l'attività originale.

Feedback errati Allarmi non necessari o fastidiosi spingono l'utente a disattivarli. Questo poi porta problemi se si vogliono ripristinare (bisogna ricordarsene) o se avviene un incidente mentre sono disattivati. Un altro problema sono i *feedback parlati*: possono essere molto precisi sul messaggio ma allo stesso tempo confusionari se in ambiente rumoroso o se sovrapposti.

2.9.3 Prevenzione

L'ideale è che un singolo errore non dovrebbe poter fare danni estesi. Alcune buone pratiche:

- **Capire** le cause dell'errore e progettare di conseguenza
- Fare **sensibility check**, ovvero controllare se l'azione rispetta il senso comune. E.g. la banca mi chiede se sono sicuro di fare un bonifico grosso
- Rendere le azioni **reversibili** o rendere difficili le azioni irreversibili
- Rendere gli errori **facili da trovare** e da correggere
- Non trattare l'azione come un errore, piuttosto aiutare la persona a completarla in maniera appropriata
- Aggiungere **constraints**, ovvero blocchi specifici per la sezione dell'azione che potrebbe causare errore. Si dividono in:
 - **Segregate control**: spargere i controlli che possono essere confusi lontani tra loro
 - **Separate modules**: i controlli non rilevanti per l'operazione corrente non sono visibili

Spesso vengono aggiunti messaggi di **richiesta di conferma** per un'azione. Questi sono però inefficaci in quanto di solito la persona ha già deciso di volerla compiere nel momento in cui preme la prima volta il pulsante. È comunque utile metterli, rendendoli più evidenti (e.g. dimensione e colore).

Note 2.9.1 (Minimizzazione degli slip). Dato che il comportamento *skilled* è **subconscio**, per ridurre gli slip non bisogna fare in modo che sia richiesta più attenzione ma piuttosto bisogna assicurarsi che le azioni e i loro controlli siano il più diversi possibili (o fisicamente distanti).

2.9.4 Mitigazione

Per mitigare gli errori bisogna aumentare il numero di controlli e ridurre il numero di punti critici in cui un errore può avvenire. I punti principali per il *design* che tiene conto degli errori sono:

- **Diffondere la conoscenza** richiesta per usare una determinata tecnologia, facendo in modo che chi la conosce bene possa lavorare efficientemente e chi non la conosce può comunque impararla
- Usare **constraints** naturali e artificiali
- Connettere correttamente i due **golfi**

2.10 User behavior patterns

Quando gli utenti interagiscono con le interfacce, il loro comportamento segue dei **pattern** che possono essere predetti. Capirli ci permette di progettare interfacce che sembrano naturali all'utente, supportando strategie e abitudini comuni e creando interazioni più intuitive.

Self Exploration Probabile che impari e si senta bene nel farlo, cliccare finestre pop-up, reinserire dati eliminati per sbaglio o mettere video in pausa. Far sì che sia possibile **esplorare** senza che questo costi per l'utente.

Instant Gratification Queste persone vogliono vedere immediatamente i risultati delle loro azioni e si sentono più fiduciosi in sé stessi. Dato che ogni esperienza positiva porta gratificazione, bisognerebbe progettare la UI in modo che la prima azione sia estremamente semplice. Le **funzionalità introduttive** non dovrebbero essere nascoste.

Satisficing Queste persone scelgono la prima cosa anche se potrebbe essere sbagliata e accettano una risposta "**sufficientemente** buona" invece della migliore se questo fa risparmiare soldi e tempo. Usare etichette brevi, semplici e veloci da leggere in modo che l'utente capisca alla prima lettura. Fornire **metodi di fuga**. Questo comportamento è spesso trovato in utenti inesperti che si trovano ad usare cose complesse.

Changes in Midstream Fornire l'opportunità alle persone di cambiare i propri obiettivi in corso d'opera, a meno che non sia possibile (wizard, hub and spoke, modal panel). Permettere all'utente di iniziare un processo, interromperlo e poi continuarlo successivamente: **reentrance**.

Deferred Choices Utenti che spesso saltano domanda necessarie, rispondendo solo al minimo necessario (per necessità o scelta), per poi tornare più tardi. Mostrare la lista breve e nascondere quella lunga, fornire valori di default e permettere di terminare successivamente.

Incremental Construction Le persone di solito non creano tutto in una volta ma partono da piccoli pezzi, ci lavorano, li testano e vanno avanti. Le interfacce devono essere **responsive** a modifiche e salvataggi veloci. I **feedback** devono essere costanti. L'obiettivo è indurre un "flow" nell'utente, in modo che il godersi l'attività sia esso stesso il premio.

Habituation L'utente non deve più ragionare per compiere azioni abituali, migliorando l'efficienza. Questo però può portare a trappole per l'utente se una *gesture* non funziona o fa qualcosa di distruttivo.

Microbreaks L'utente fa piccole pause tra attività di concentrazione. Queste sono fondamentali per i processi cognitivi e per rilassarsi dallo stress. Le interfacce devono supportare un'interruzione e ripresa delicata e piacevole.

Spatial Memory Spesso le persone trovano gli oggetti ricordandosi dove sono e non come sono chiamati. Posizionare i controlli in posti prevedibili, fornire spazi adattabili dall'utente. L'inizio e la fine delle liste e dei menù sono posti speciali da un punto di vista cognitivo.

Prospective Memory Dare all'utente strumenti per creare i propri metodi di promemoria (e.g. lasciare le cose davanti alla porta per prenderle il giorno dopo). Lasciare artefatti in giro che permettano di identificare attività incomplete.

Streamlined Repetition Gli utenti a volte devono fare la stessa azione ripetutamente: gli va resa facile per evitare di sprecare tempo.

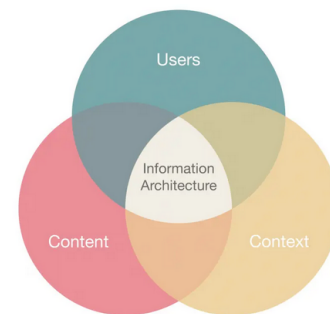
Keyboard Only Collegato a *habituation* e *flow*, rende più facili attività ripetitive fornendo comandi da tastiera che sono più veloci del mouse.

Other People's Advice Le persone influenzate da consigli ed esperienze di colleghi, che siano questi diretti o indiretti. È importante la presenza di una **community**.

2.11 Information architecture

La IA si concentra sull'**organizzare**, **strutturare** ed **etichettare** il contenuto in maniera efficace e sostenibile per aiutare l'utente a trovare le informazioni e completare le attività. Per creare dei sistemi di informazione è quindi necessario capire la natura di:

- **Users:** l'audience, le attività, i bisogni
- **Context:** obiettivi di business, investimenti, politica, cultura, tecnologia, risorse, limitazioni
- **Content:** obiettivi di contenuto, tipi di documenti e dati, volume, strutture preesistenti, comando e proprietà



I componenti principali della IA sono:

- **Organization Schemes and Structures:** come categorizzare e strutturare le informazioni
- **Labeling Systems:** come rappresentare le informazioni
- **Navigation Systems:** come gli utenti esplorano o si muovono all'interno delle informazioni
- **Search Systems:** come gli utenti cercano le informazioni

2.11.1 Organization Schemes

Descrivono come **categorizzare** il contenuto e i diversi modi in cui creare **relazioni** tra i vari componenti. Gli schemi si dividono in:

- **Exact:** dividono le informazioni oggettivamente in categorie mutualmente esclusive. Sono utili per gli Information Architects ma possono essere difficili per gli utenti. Alcuni esempi sono:
 - **Alfabetici:** è importante che le etichette corrispondano a ciò che l'utente cerca
 - **Cronologici:** fondamentale che ci sia un accordo su come datare l'informazione
 - **Geografici**
- **Subjective:** categorizzano le informazione in un modo che può essere specificato o definito dall'organizzazione o dal campo. Sono più difficili da progettare ma spesso più utili degli *exact*. Alcuni esempi:
 - **Argomento**

- **Task:** considera le necessità, azioni, domande o processi che l'utente può avere per quel contenuto
- **Audience:** organizzare i contenuti in base al segmento di utenza. Possono essere *chiusi* o *aperti* se ci si può spostare o meno. E.g. parental control
- **Metafore:** aiuta gli utenti associando i contenuti a concetti familiari, e.g. icone

Note 2.11.1 (Ibridi). È possibile mischiare i diversi tipi di schemi ma è sconsigliato in quanto potrebbe causare confusione agli utenti. Di solito si fa quando il team non riesce a giungere ad un accordo su quale usare.

2.11.2 Organizational Structures

Descrive come è definita la **relazione** tra diversi pezzi del contenuto e permette all'utente di predire dove troverà le informazioni. Le quattro principali strutture sono:

- **Gerarchico:** spesso chiamata ad albero o hub-and-spoke, utilizza un approccio padre-figlio tra i diversi pezzi di informazione. L'utente parte da categorie generali e scende in quelle sempre più dettagliate
- **Sequenziali:** richiedono che l'utente proceda passo passo seguendo un percorso specifico attraverso il contenuto, e.g. processo di acquisto online
- **Matrici:** permettono all'utente di scegliere il loro percorso dato che il contenuto è collegato in molti modi, e.g. la struttura del web
- **Database:** utilizza un approccio dal basso all'alto dove il contenuto dipende fortemente dai collegamenti tra i suoi metadati. Facilita un'esperienza dinamica e permette filtri avanzati e possibilità di ricerca

È fondamentale che gli architetti tengano presente che la struttura scelta deve tollerare miglioramenti e futuri aggiornamenti, in particolare:

- Permettere la **crescita**: si deve poter aggiungere nuovo contenuto e nuove sezioni
- Evitare strutture troppo o troppo poco **profonde**: nel primo caso servono menù giganteschi mentre nel secondo l'informazione è sepolta sotto troppi livelli

2.11.3 Navigation

La navigazione riguarda come l'utente si **orienta** all'interno dell'interfaccia e mira a garantire che l'utente sia capace di trovare l'informazione o la funzionalità che sta cercando. Si basa su due principi fondamentali:

- **Findability:** essere capaci di localizzare l'informazione che l'utente considera rilevante (e.g. biblioteca). È importante piazzare gli elementi con consistenza nell'interfaccia
- **Discoverability:** la possibilità che gli utenti hanno di scoprire nuove informazioni o funzionalità di cui non erano a conoscenza (e.g. libro sconosciuto in biblioteca). Si possono piazzare ad esempio i nuovi contenuti vicino a quelli più popolari. Di solito però l'interfaccia fornisce strumenti che permettono di scoprire nuovi contenuti in maniera indiretta, per poi rendersene conto solo dopo.

2.11.4 Componenti

Durante la progettazione dell'interfaccia è importante rimanere **consistenti** e **prevedibili** per la scelta degli elementi. Anche se gli utenti non lo sanno, hanno sviluppato una familiarità per come gli elementi si comportano e bisogna quindi scegliere di conseguenza, mirando all'efficienza e alla soddisfazione. Alcuni esempi sono:

- **Input Controls:** e.g. checkboxes, radio buttons, dropdown lists, list boxes, bottoni, toggles, campi di testo, date field

- **Navigational Components:** e.g. breadcrumbs, slider, barra di ricerca, paginazione, tags e icone
- **Informational components:** e.g. tooltips, icone, barre di caricamento, notifiche e messaggi
- **Containers:** fisarmonica

2.12 Innovazione

L'innovazione è comunemente definita come l'introduzione di nuove combinazioni, come beni, metodi produttivi, nuovi mercati, etc. . . Per quanto esistano definizioni diverse, in tutte dominano la **novità**, il **miglioramento** e la **diffusione**. Possiamo dire quindi che è un qualcosa di **originale** e **più efficace** che irrompe nel mercato o nella società.

Note 2.12.1 (Invenzione). L'innovazione è correlata ma diversa dall'invenzione: la prima è più diretta all'implementazione pratica della seconda per avere un impatto su un mercato o società. Non tutte le innovazioni hanno bisogno di una nuova invenzione.

Innovazione sostenuta La maggior parte dei processi di innovazione si basano su miglioramenti **incrementali** di prodotti già esistenti. Avvengono un passo alla volta, senza bisogno di cambiamenti nella compagnia, nelle abilità necessarie e rimanendo lenti e a basso rischio, mantenendo il target e il mercato stabile.

Innovazione dirompente È un'innovazione che crea un nuovo mercato e catena di valore, scuotendo quelli esistenti e spiazzando le compagnie, prodotti e alleanze già esistenti. Ad oggi indica avanzamenti tecnologici e scientifici dirompenti. Non tutte le innovazioni sono dirompenti, anche se rivoluzionarie, e.g. la macchina.

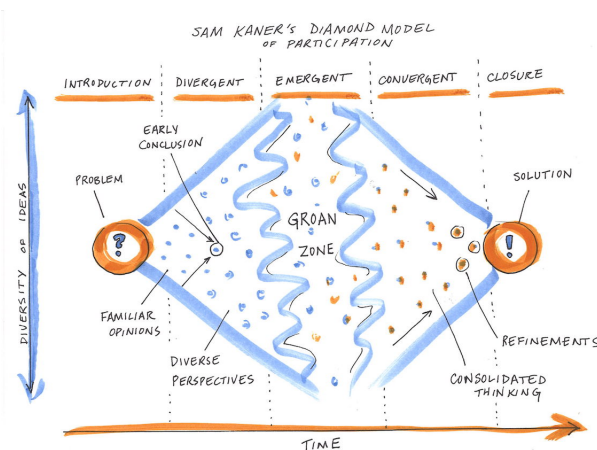
È importante anche distinguere il product **management** dal product **development**: il primo riguarda il **cosa** mentre il secondo il **come**. Il team di sviluppo riceve i requisiti specificati dal manager e li usa per creare un prodotto funzionante che rispecchi gli standard della compagnia. Consiste quindi nel tradurre un concetto in qualcosa di vendibile sul mercato.

2.12.1 Design Methods

Human Centered Design Process L'azienda **IDEO** è una delle più innovative del mondo. Il loro punto chiave è l'empatia per l'utente finale. Credono che la chiave per capire cosa gli umani vogliono veramente risieda in:

- **Osservarne il comportamento:** provare a capire le persone osservandole
- Mettersi nella **stessa situazione** dell'utente finale, per capire come gli utenti si sentono

IDEO definisce HCD Process come un approccio creativo al problem-solving che parte con le persone e finisce con soluzioni fatte apposta per soddisfare le loro necessità.



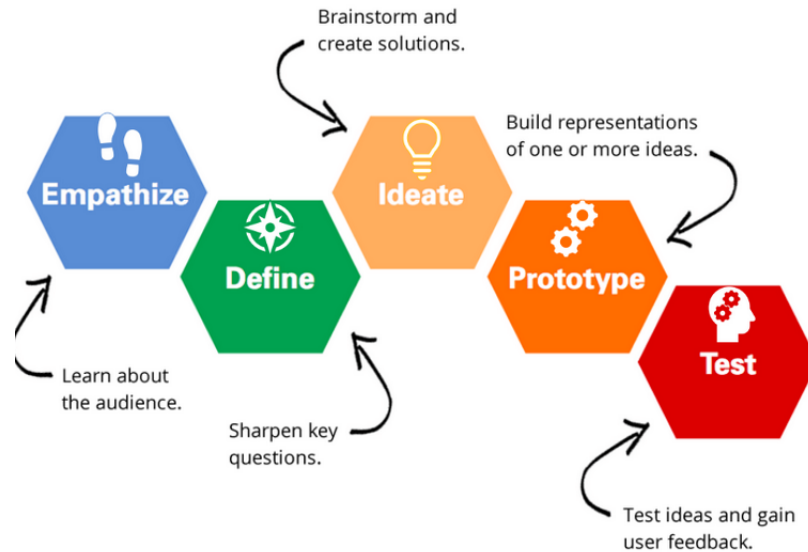
Il processo HCD si divide in tre fasi:

1. **Inspiration:** capire i bisogni degli utenti e le loro difficoltà, dimenticando i preconcetti che si hanno. Bisogna essere aperti a tutte le possibilità.
2. **Ideation:** dopo aver consolidato le ricerche, bisogna visualizzare e discutere tutte le possibili soluzioni, anche se assurde. È fondamentale ricevere feedback e ripartire partendo da quelli in modo da ottenere idee buone. Non serve creare prototipi ma è sufficiente fare sketch o modellini.
3. **Implementation:** in questa fase si crea un prototipo che possa essere provato dagli utenti e la produzione dell'oggetto stesso. Si inizia anche a creare un modello di business.

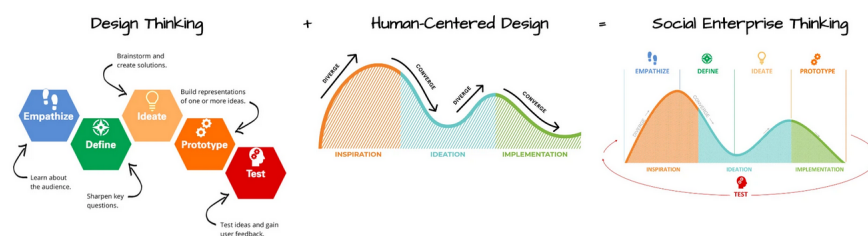
Design Thinking Il Design Thinking è un processo **iterativo** dove si cerca di capire l'utente, mettere in discussione le assunzioni e ridefinire i problemi nel tentativo di trovare strategie e soluzioni alternative che potrebbero non essere ovvie. Allo stesso tempo fornisce un approccio **solution-based** per risolvere i problemi.

È estremamente utile per quei problemi che sono mal definiti o sconosciuti, permettendo di ridefinirli con un approccio **antropocentrico** tramite sessioni di **brainstorming** e approcci **hands-on** per quanto riguarda prototipi e test. Si divide in cinque fasi:

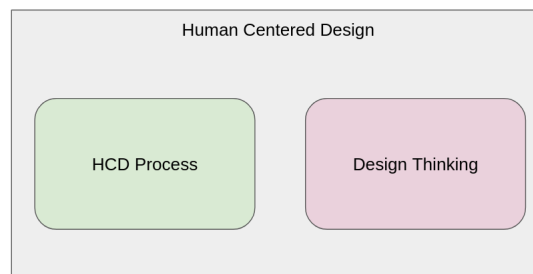
1. **Empatizzare:** studiare i propri utenti
2. **Definire** i bisogni degli utenti, i problemi e le tue intuizioni
3. **Ideare** sfidando le assunzioni e creando idee per soluzioni innovative
4. **Prototipo**
5. **Test**



La combinazione di questi metodi di design porta al **Social Enterprise Thinking**.



Note 2.12.2 (HCD). Applicare lo Human-Centered Design sopra al Design Thinking permette di costruire una soluzione che sia capace di fare soldi e che allo stesso tempo soddisfi le necessità delle persone che la utilizzano. È importante notare che HCD è un **mindset** mentre HCD Process e Design Thinking sono **design methods**.



2.12.2 Development methods

Waterfall È una metodologia di sviluppo lineare, dove i requisiti dei clienti e degli stakeholders sono raccolti all'inizio del progetto e poi viene creato un piano sequenziale per soddisfarli.

Agile È un termine ombrello per un insieme di framework e pratiche basate sui valori e principi espressi nel "Manifesto for Agile Software Development". La differenza principale dagli altri approcci è il concentrarsi sulle **persone** che lavorano e su **come lavorano assieme**. È il metodo migliore per HCD e Design thinking dove iterazioni continue sono fondamentali.

Scrum È un framework agile progettato per piccoli gruppi che dividono il loro lavoro in obiettivi che possono essere completati in breve tempo (circa due settimane), chiamati **sprint**. Il team controlla i progressi in riunioni giornaliere di quindici minuti, chiamati **daily scrums**. Alla fine dello sprint si fa una **review** per mostrare il lavoro fatto e migliorare per i prossimi. Uno dei principi fondamentali è che i clienti **cambieranno idea** su ciò che vogliono o di cui hanno bisogno e ci saranno quindi sfide imprevedibili per cui un approccio pianificato non funzionerebbe. Scrum usa quindi un approccio empirico basato su prove, accettando che il problema non può essere compreso e definito completamente subito all'inizio del progetto. Si concentra piuttosto sull'abilità di rispondere velocemente a nuovi requisiti e di adattarsi a nuove tecnologie e cambiamento del mercato. Si compone di tre ruoli:

- **Product owner**, rappresenta gli stakeholder e i clienti ed è responsabile per consegnare dei buoni risultati di business. Di solito definisce il prodotto in termini "customer-centric" e li aggiunge al **product backlog** in ordine di importanza e dipendenze
- **Development team**
- **Scrum master**: non il classico capo ma agisce da buffer tra il team e ogni distrazione, facendo in modo che il framework sia seguito

3 User Experience

3.1 Wireframing

Definizione 3.1.1 (Wireframe). *I wireframe sono **blueprint** che migliorano la comunicazione tra designer e programmatori riguardo la struttura del sito web o app che stanno sviluppando.*

Servono ad evitare che le varie persone coinvolte nel progetto prendano strade diverse seguendo solo ciò che pensano sia meglio individualmente.

L'attività di **wireframing** è quindi il processo di creazione di sketch basilari dell'interfaccia di un sito o un'applicazione per dimostrare la sua **struttura** e il suo **layout**.

Esistono due tipi di wireframe:

- **Low-fidelity**: semplici sketch dell'interfaccia che si concentrano su struttura e layout
- **High-fidelity**: rappresentazione dettagliata dell'interfaccia con topografia di base, icone e dettagli di documentazione

3.2 Adaptability

Il contesto gioca un ruolo fondamentale nel UX Design (e.g. utente semplice da telefono contro esperto da computer) ed è quindi importante fare **ricerche sugli utenti** e **testing** in modo da comprendere a fondo i vari modi in cui gli utenti possono interagire.

Definizione 3.2.1 (Adaptability). *Un termine ombrello nel UX Design che si riferisce alla capacità di un design di adattarsi e rispondere alle diverse necessità, contesti, capacità e preferenze degli utenti.*

3.2.1 Dispositivo

Responsive Design Con sempre più dispositivi venduti, ognuno con diversi layout e standard, è fondamentale fare in modo che i siti web siano leggibili ed utilizzabili ovunque. Il responsive design è un metodo di designing e programmazione di pagine web e app per **adattarle** a *dimensione* e *risoluzione* dei diversi dispositivi.

Il responsive design permette all'utente di avere un'esperienza **consistente** indipendentemente dal dispositivo e rende più facile **manutenere** e **aggiornare** i siti web, dato che va fatto su una sola versione. Esistono alcune tecniche:

- **Flexible grid**: la pagina è divisa in colonne di larghezza uguale e ogni elemento può usarne una o più. Si possono impostare dei breakpoint e degli spazi. Si divide in diversi *pattern*:
 - **Tiny Tweaks**: il layout rimane pressoché lo stesso su tutti i dispositivi e lavora su una colonna, cambiando solo dimensione del font, spaziature e margini
 - **Column Drop**: si parte dai dispositivi mobili con una singola colonna e se ne aggiungono man mano che lo schermo cresce, mentre scendono sotto alla precedente per quelli piccoli
 - **Mostly Fluid**: la griglia diventa fluida, con i margini che aumentano con il crescere dello schermo e con una larghezza massima per evitare che le righe diventino troppo lunghe
 - **Layout Shifter**: il layout cambia significativamente ai breakpoint
 - **Off Canvas**: la navigazione o il contenuto secondario vengono nascosti di default sui dispositivi mobili e si possono mostrare tramite menu o gesture
- **Flexible images**: le immagini scalano e si aggiustano in base alla dimensione dello schermo
- **Media query e breakpoints**: permettono l'applicazione di stili diversi in base alle caratteristiche del dispositivo

Alcune buone pratiche sono:

- **Keep it simple**: evitare layout complessi e mantenere un design minimale

- Priorizzare il **contenuto**: deve essere la parte più importante della pagina e facilmente leggibile da tutti i dispositivi
- Design per il **touch**: assicurarsi che i bottoni siano grandi abbastanza per essere toccati con un dito
- Ottimizzare per il **tempo di caricamento**, in particolare immagini o altri oggetti pesanti
- **Test** su tanti tipi di dispositivi per verificare che tutto appaia e funzioni correttamente

Mobile first È una filosofia che priorizza i dispositivi mobili rispetto a quelli desktop o comunque con schermi più grandi. Questo implica che i designer si assicurano che il contenuto e le funzionalità più importanti siano facilmente accessibili sui piccoli schermi e che sia facile navigare il sito con controlli di tipo touch. Inoltre questo approccio si assicura che il sito carichi velocemente sui dispositivi con una connessione più lenta.

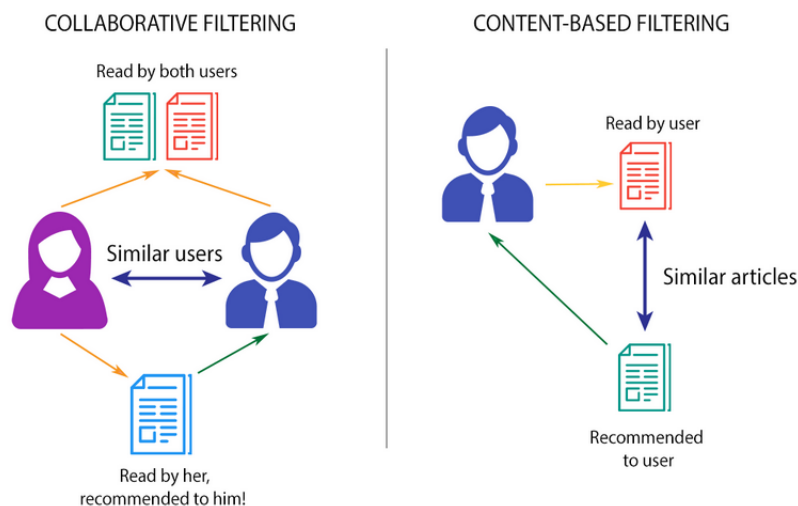
Si basa su un design di tipo **progress advancement**, ovvero che parte dalle funzionalità minime e interazioni basilari (per mobile) per poi aggiungerne di più complesse (per desktop). Si oppone alla **graceful degradation**.

Adaptive Design Il design adattivo consiste nel creare diversi tipi di layout per ogni diverso dispositivo. È più costoso del *responsive design* ma a volte è migliore, e.g. grandi aziende che non vogliono riscrivere il codice già presente per desktop.

3.2.2 Utente

Adaptive UX Per far sì che un sistema si adatti all'utente è importante avere informazioni su di lui, ad esempio:

- **Collaborative filtering**, e.g. se ad A piace X e Y e a B piace X allora a B potrebbe piacere Y
- **Content-based filtering** che si concentra invece su contenuti simili a qualcosa che piace ad un utente



Accessibilità È un aspetto fondamentale del UX Design perché assicura che tutti gli utenti possano accedere e interagire con i prodotti digitali indipendentemente dalle loro abilità. Non solo il 15% della popolazione mondiale ha una qualche forma di disabilità, ma bisogna anche tenere in considerazione le compromissioni temporanee (e.g. braccio rotto). In diversi paesi è obbligatorio per legge (e.g. EU). Lo standard è il **Web Content Accessibility Guidelines**, ad oggi alla versione 2.2, che si compone di quattro principi (POUR):

- **Perceivable**: le informazioni devono essere presentate in modi che l'utente possa percepire
- **Operable**: i componenti dell'interfaccia devono essere utilizzabili
- **Understandable**: le informazioni e le operazioni devono essere chiare
- **Roubst**: il contenuto deve funzionare con tecnologie di supporto

Ognuno è poi diviso in tredici linee guida, ognuna con i suoi criteri, che portano a tre livelli di conformità:

- **A**: livello minimo, se non raggiunto alcuni utenti sono completamente bloccati
- **AA**: quello raccomandato e spesso obbligatorio per legge
- **AAA**: il migliore, non sempre raggiungibile a seconda del contenuto

Le principali linee guida per il livello *AA* sono:

- *Contrasto* 4.5 : 1 per il testo e 3 : 1 per i componenti dell'interfaccia
- Accesso tramite *tastiera* per tutte le funzionalità
- *Testo alternativo*, ovvero descrizioni significative per tutte le immagini che non siano decorative
- *Semantica HTML* che usi gli heading appropriati, le label per i form di input e ARIA quando necessario
- *Consistenza*: navigazione nello stesso ordine, funzionalità uguali chiamate in maniera identica
- *Testo ridimensionabile* fino al 200% senza perdere funzionalità

3.3 Wireflows

A differenza dei *wireframe*, che sono statici, gli **user flow** aiutano a descrivere le possibili **azioni** che un utente può fare nell'interfaccia. Aiuta a mappare tutti i possibili passaggi e movimenti dell'utente nell'applicazione o sito web.

Partendo dalle *personas*, i *requisiti* e gli *scenari* si pensa a tutti i possibili punti di contatto che l'utente avrà con il sistema per definire così un **diagramma di flusso**. Questo può essere poi integrato con i *wireframe* per dare vita ai **wireflows**.

3.4 Mockups

Definizione 3.4.1 (Mockup). *Un mockup è una rappresentazione ad alta fedeltà del prodotto che include più dettagli visivi e elementi di design di un wireframe, permettendo di testare e rifinire come il prodotto si presenta.*

I mockup vengono di solito utilizzati per comunicare i concetti di design e le funzionalità in maniera più efficace. Sono anche utili per alcuni tipi di studi sull'usabilità dove serve il feedback sull'aspetto generale del design.

Note 3.4.1. È fondamentale notare che un mockup NON ha le funzionalità necessarie per fare test significativi od ottenere feedback profondi in quanto non possiede alcun elemento interattivo.

3.5 Pretotyping

3.5.1 Mercato

Così come in legge un imputato è innocente fino a prova contraria, nel mercato dovremmo assumere che un prodotto sia fallito fino a prova contraria. Questo perché statisticamente solo il 10% dei prodotti ha successo.

Definizione 3.5.1 (Market failure). *La maggior parte dei nuovi prodotti fallirà sul mercato, anche se sono eseguiti in maniera competente.*

Il prototyping fornisce strumenti e tecniche necessarie a validare un'idea con poche risorse e in poco tempo prima di investire per lo sviluppo.

Per minimizzare il rischio di falsi negativi o positivi sul mercato è necessario partire da "thoughtland" per ottenere opinioni

Osservazione 3.5.1 (Translation problem). Un'idea è un'astrazione soggettiva che ci si immagina nella propria testa. Nel momento in cui la si prova a comunicare c'è un problema di "traduzione", soprattutto se è molto innovativa.

Osservazione 3.5.2 (Prediction problem). Anche se gli interlocutori hanno un'idea astratta della tua idea che è molto simile a come la immagini, gli umani sono pessimi a predire se vogliono veramente qualcosa che non hanno mai sperimentato e come la userebbero.

per poi arrivare ad "actionland" per ottenere dati.

3.5.2 Pretotype

Prima ancora di creare un *prototipo* è importante capire se fallirà o meno sul mercato prima di investire tempo e denaro. Questo si fa con un **pretotype**. I passaggi per lo sviluppo sono:

1. Isolare l'**assunzione chiave** e capire se è vera o falsa
2. Scegliere un tipo di **pretotype** che permetta di isolare l'assunzione
3. Fare un'ipotesi di **interesse di mercato**: quante persone e di che tipo vorrebbero il pretotype
4. **Test** del pretotype mettendolo nel mondo reale e analizzando come le persone ci interagiscono
5. **Imparare** dai risultati del test per **rifinire** il pretotype. Se l'ipotesi permane, testare in nuove situazioni (**hypozoom**)

Fake Door Questo metodo consiste nel pubblicizzare l'idea del prodotto che ancora non esiste fingendo il contrario e vedere quante persone sarebbero interessate. Si può fare tramite pubblicità, siti web, form, email o poster.

Mechanical Turk Fingere che un algoritmo sia implementato facendo in realtà fare il lavoro ad un umano e vedere se interessa.

Impersonator Utilizzare un prodotto già esistente con un'interfaccia diversa.

Pinocchio Creare una finta versione del prodotto che non funziona per verificare ad esempio se è il giusto form factor, ha le giuste funzionalità ed è usabile. E.g. finto telefono di plastica.

One Night Stand Fare una versione completa dell'esperienza che si vuole creare ma solo per una volta per capire se può funzionare.

Facade Noleggiare o chiedere in prestito attrezzature, spazi e materiali per simulare un'infrastruttura che in realtà non esiste, evitando così di fare subito un investimento grosso.

3.6 Prototyping

Nel momento in cui si ha imparato tutto il possibile sulla domanda di mercato dai *pretotypes* ed è necessario mettere il vero prodotto nelle mani dei clienti per testarlo e rifinirlo, si crea un **prototipo**.

Definizione 3.6.1 (Prototipo). *Un primo esempio sperimentale di un design che permette all'utente di visualizzare e **interagirci** prima che il prodotto finale sia sviluppato.*

Definizione 3.6.2 (Prototyping). *Il processo con il quale i team di design ideano, sperimentano e portano in vita i concetti. È la fase in cui le idee sono implementate in un qualcosa di tangibile.*

Un prototipo porta quindi a diversi vantaggi:

- Fornisce delle fondamenta solide per l'ideazione e dà agli stakeholder una chiara immagine dei potenziali benefici, rischi e costi
- I cambiamenti possono essere fatti all'inizio del processo, prima del rilascio, evitando errori costosi e il dover rimanere per forza attaccati ad una versione
- Il feedback dell'utente permette di identificare cosa funziona meglio e se è necessario rifare delle parti

3.6.1 Fidelity

Definizione 3.6.3 (Fidelity). *Quanto un prototipo assomiglia al prodotto finale in termini di aspetto e funzionalità.*

Esistono due livelli di fedeltà di un prototipo:

- **Low-fidelity:** di solito molto economici in termini di soldi e tempo, fornisce interazioni molto semplici (se non inesistenti). Facile da modificare. È poco realistico e potrebbe essere troppo basilare perché un utente riesca a dare un feedback, senza contare che lo costringe a dover immaginare come sarebbe il prodotto.
- **High-fidelity:** più costosi e lunghi da creare, sono spesso sviluppati in ambienti specifici (e.g. Figma) e danno una sensazione di prodotto finito, rendendoli attraenti sia per gli utenti che per gli stakeholder. I test forniscono dati più accurati anche se gli utenti potrebbero soffermarsi troppo su dettagli superficiali invece che il contenuto.

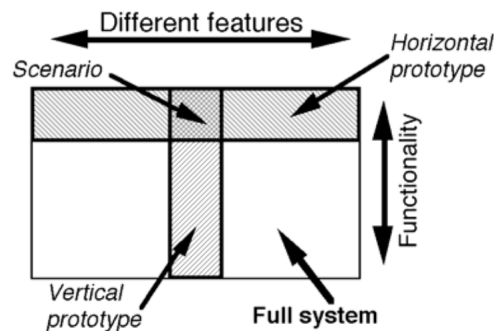
Ed eventualmente tutte le possibilità che ci sono in mezzo (**mid-fidelity**), ad esempio un wireframe avanzato con solo alcune parti interattive. In generale la fedeltà si basa su sei dimensioni:

- **Realismo fisico/visuale:** quanto simile è il prototipo al prodotto finale per quanto riguarda l'estetica e il design visivo
- **Obiettivo:** l'ampiezza e profondità del design rappresentato nel prototipo
- **Funzionalità:** cosa funziona effettivamente nel prototipo, e.g. bottoni in una web app
- **Dati:** se il prototipo funziona con dati reali o finti
- **Autonomia:** se il prototipo funziona da solo o ha bisogno di input dal designer o dall'utente
- **Piattaforma:** se il prototipo è un interim o l'implementazione finale

3.6.2 Scope

Lo *scope* descrive quanto l'utente sarà in grado di vedere e testare. Si divide in:

- **Orizzontale:** fornisce una visione completa del sistema senza però andare troppo nel dettaglio per ogni funzionalità. È quindi limitato ad un solo layer. È utile agli stadi iniziali del design in quanto permette agli stakeholder di identificare velocemente potenziali problemi.
- **Verticale:** fornisce una visione dettagliata di una particolare specifica, permettendo di testare se il prodotto è fattibile o se un particolare requisito è soddisfatto



3.6.3 Tecniche

Carta Un prototipo di carta è *low-fidelity* e rappresenta il layout e le funzionalità basilari di un prodotto o servizio. È veloce ed economico. Consiste nel disegnare su carta o cartone e poi l'utente può interagirci fisicamente. Può essere usato per testare agli stadi iniziali dove posizionare bottoni e menu per il flusso delle informazioni e le interazioni dell'utente.

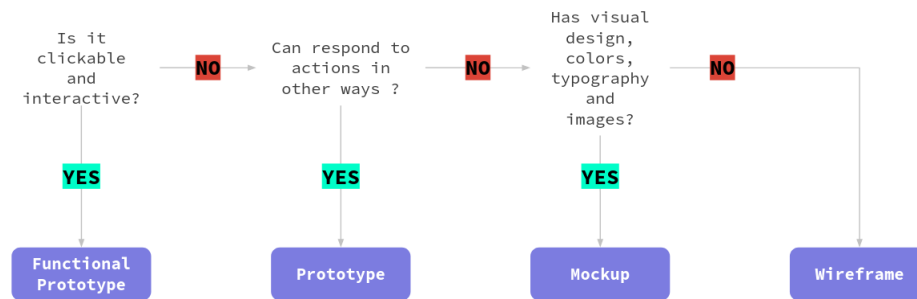
Wireframe È la versione digitale di quello di carta.

Mechanical Turk Un prototipo che simula il comportamento del prodotto usando in realtà input umani invece della tecnologia. Ad esempio può consistere in audio o video preventivamente registrato, risposte programmate o interazioni live con l'utente.

Funzionali Prototipo che include funzionalità funzionanti o parzialmente funzionanti per testare gli aspetti tecnici di un prodotto, come reattività o funzionamento. Utile negli stadi finali del design quando si vogliono perfezionare dettagli tecnici prima di passare allo sviluppo completo.

Non funzionali Un prototipo che sembra esattamente come il prodotto reale ma non ha nessuna funzionalità. È utilizzato per dimostrare il design fisico e l'interfaccia utente senza dover programmare in maniera dettagliata.

Osservazione 3.6.1. La differenza tra i prototipi e i *mockup* e *wireframe* è che i primi sono versioni più avanzate creati sulla base degli ultimi che permettono di testare e validare il design prima della fase di sviluppo.



3.7 Design systems

Con la crescita di un prodotto più designer prendono decisioni diverse e i componenti vengono reimplementati in maniera leggermente differente. Un po' alla volta le **inconsistenze** si accumulano e la qualità scende.

Definizione 3.7.1 (Design System). *Un insieme di componenti riutilizzabili, guidati da standard precisi, che possono essere assemblati per costruire applicazioni. Include:*

- Libreria dei componenti, e.g. bottoni
- Design tokens, e.g. colori e font
- Linee guida per l'utilizzo e documentazione
- Principi di design e pattern
- Implementazioni di codice

I sistemi di design garantiscono:

- **Consistenza:** gli stessi componenti si comportano allo stesso modo ovunque
- **Efficienza:** crea una volta e riusa molteplici

- **Qualità:** accessibilità e responsività
- **Scalabilità:** nuovi membri del team si possono unire più facilmente
- **Manutenibilità:** risolvere un errore lo fa per tutti

3.7.1 Atomic Design

Questo metodo si compone di blocchi da costruzione uno più grande dell'altro:

- **Atomi:** elemento basilare che non può essere diviso ulteriormente, e.g. bottoni
- **Molecole:** combinazioni di atomi che funzionano assieme, e.g. form di ricerca (bottone, input e label)
- **Organismo:** componenti complessi dell'interfaccia composti dai precedenti, e.g. header
- **Template:** layout di pagina che mostrano la struttura del contenuto, e.g. homepage
- **Pagine:** istanze specifiche di layout con vero contenuto

3.7.2 Design Tokens

Definizione 3.7.2 (Design Token). *Decisione di design salvata come dato.*

Permette di cambiare una volta per tutte le occorrenze e indipendentemente dalla piattaforma. E.g. definire il colore primario.

3.8 Evaluation

Una volta creato un *prototipo* lo si vuole **testare** con gli utenti. Questo può portare a trovare problemi di **usabilità**, possibile margine di miglioramento e capire come gli utenti si comportano e cosa preferiscono.

3.8.1 Definizioni

Esistono due definizioni diverse di *usability* che si concentrano su due aspetti leggermente diversi.

Nielsen

Definizione 3.8.1 (Usability - Nielsen). *L'usabilità è un attributo di qualità che definisce quanto facili le interfacce utente sono da utilizzare. Si compone di:*

- **Learnability:** quanto è facile compiere azioni basilari per la prima volta
- **Efficiency:** una volta imparato il design, quanto veloce si eseguono le azioni
- **Memorability:** quando gli utenti ritornano sul prodotto dopo tanto tempo, quanto è facile riprendere l'utilizzo
- **Errors:** quanti errori compie l'utente, quanto sono gravi e quanto è facile riprendersi da essi
- **Satisfaction:** quanto è piacevole usare il design

ISO La definizione ISO si concentra invece sul fatto che l'usabilità sia il risultato dell'interazione con il sistema: i vari attributi possono aiutare ma l'usabilità non è una caratteristica intrinseca del prodotto.

Definizione 3.8.2 (Usability - ISO9241-11). *L'usabilità è il grado a cui un sistema, prodotto o servizio permette agli utenti di compiere determinate azioni con **efficacia**, **efficienza** e **soddisfazione** in uno specifico contesto d'uso.*

Definizione 3.8.3 (Efficacia). *Accuratezza e completezza con le quali gli utenti raggiungono obiettivi specifici. Si può misurare in:*

- Percentuale di task completate con successo
- Tasso di errore
- Accuratezza nelle decisioni, e.g. percentuale di selezioni corrette
- Completezza dei risultati

Definizione 3.8.4 (Efficienza). *Risorse utilizzate in relazione ai risultati ottenuti, e.g. tempo, impegno umano, costi o materiali. Si misura in:*

- Mediana o percentile del tempo speso per ogni task completata con successo
- Throughput delle task per ora o passi necessari per completarla
- Impegno umano valutato tramite scale di workload, e.g. NASA-TLX, SWAT o MRQ
- Costo per risultato corretto quando rilevante

Definizione 3.8.5 (Soddisfazione). *Quanto le risposte fisiche, cognitive ed emotive di un utente raggiungono le sue necessità e aspettative. Si misura in:*

- **System Usability Scale** per l'usabilità generale percepita
- **UMUX-Lite** per l'utilità/usabilità percepita quando SUS è troppo lungo
- Scale specifiche di usabilità come **Chatbot Usability Scale**

Note 3.8.1. Conviene separare la soddisfazione (soggettiva) dall'efficacia/usabilità (oggettiva) ma analizzarle assieme per provare a spiegare le divergenze, e.g. veloce ma frustrante.

Utile

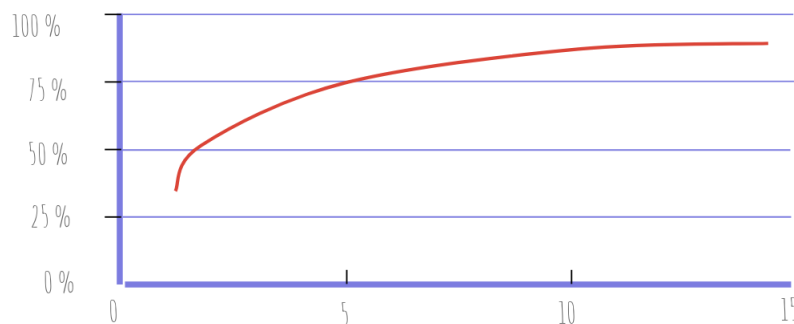
Definizione 3.8.6 (Utilità). *Quanto un prodotto riesce a fare ciò di cui gli utenti hanno bisogno.*

La combinazione di *utilità* e *usabilità* indica che un prodotto è **utile**.

3.8.2 Valutazioni euristiche

La valutazione euristica è un metodo di valutazione delle interfacce utente basato su principi di usabilità ben definiti (*euristiche*). Di solito comprende un piccolo gruppo di valutatori, circa 3 – 5. Anche se questo metodo non fornisce un approccio sistematico alla risoluzione dei problemi di usabilità, può **guidare** la creazione di design rivisitati basati sui principi violati trovati in maniera economica.

PROPORTION OF USABILITY PROBLEMS FOUND



Nielsen ha identificato dieci euristiche per l'usabilità:

1. **Visibilità dello stato del sistema:** i design dovrebbero mantenere l'utente informato riguardo a ciò che sta accadendo tramite feedback appropriati
2. **Mappare il sistema al mondo reale:** il design dovrebbe usare parole, frasi e concetti familiari per l'utente
3. **User control e libertà:** gli utenti necessitano di una "uscita di emergenza" evidente per poter abbandonare l'azione non voluta
4. **Consistenza e standard:** seguire le convenzioni della piattaforma per evitare che l'utente debba provare a indovinare il significato delle cose
5. **Prevenzione degli errori:** prima ancora di avere dei messaggi di errore ben strutturati è importante provare ad evitare che accadano
6. **Recognition invece di recall:** ridurre al minimo il carico sulla memoria degli utenti facendo in modo che gli elementi, le azioni e le opzioni siano visibili
7. **Flessibilità ed efficienza d'uso:** garantire shortcut che un utente esperto può sfruttare
8. **Estetica e design minimal:** evitare informazioni irrilevanti nelle interfacce
9. **Riconoscere, diagnosticare e riprendersi dagli errori:** i messaggi di errore dovrebbero essere frasi che indicano precisamente il problema e suggeriscono una soluzione costruttiva, invece che un codice
10. **Aiuto e documentazione:** per quanto idealmente il design non dovrebbe aver bisogno di ulteriori spiegazioni, potrebbe servire della documentazione da fornire all'utente per completare certi task

3.8.3 Cognitive Walkthrough

Metodo di valutazione che si basa sull'idea che l'utente impari ad usare l'interfaccia tramite l'**esplorazione**, aiutato dai suoi **obiettivi** e utilizzando semplice **ragionamento**.

La valutazione viene quindi fatta da un **esperto** che compie un insieme di task imitando le performance di un utente. Prima che l'esperto possa procedere, deve ricevere dal designer o sviluppatore:

- **Descrizione utente**, che include il suo livello di esperienza con i computer e le assunzioni del designer
- **Descrizione sistema** che include le operazioni e le performance
- **Descrizione task** che specifica ciò che l'esperto dovrà fare seguendo il punto di vista dell'utente
- **Sequenza azioni** che descrive ciò che il sistema deve mostrare e le azioni che l'utente deve compiere per completare una task. La combinazione di un system display e un'azione utente compongono uno *step*

Durante ogni step della *sequenza azioni*, l'esperto deve porre le seguenti domande:

- A questo stadio, il prossimo obiettivo è chiaro?
- L'azione corretta è ovvia?
- È chiaro che questa azione porta all'obiettivo?
- Quali problemi ci sono nel compiere questa azione?

3.8.4 Usability testing

Processo nel quale un *ricercatore* chiede ad un *partecipante* di compiere delle task, di solito usando una o più interfacce, mentre il suo comportamento viene **osservato** e ascoltato per eventuali **feedback**. Abbiamo quindi tre elementi:

- **Facilitator:** il ricercatore che guida il *partecipante* durante il test. Deve dare istruzioni, rispondere alle domande e farne in seguito. Il suo lavoro è assicurare che i risultati siano validi e di alta qualità senza influenzare il comportamento del *partecipante*.
- **Task:** attività realistiche che il *partecipante* potrebbe dover compiere nella vita reale; possono essere molto specifiche o molto generali in base alle domande del ricercatore e al tipo di test. Possono essere comunicate verbalmente o in forma scritta ma è importante che siano precise al massimo in quanto il minimo errore può influenzare il *partecipante*.
- **Partecipante:** utente realistico del prodotto o servizio; può essere qualcuno che lo usa già o comunque che ha un simile background o necessità. Idealmente la scelta parte dalle *personas* definite. A volte gli viene chiesto di pensare ad alta voce.



Esistono due tipi di test: **quantitativo** e **qualitativo**.

	Ricerca Qualitativa	Ricerca Quantitativa
Domande a cui risponde	Perché?	Quanti e quanto?
Obiettivi	Sia <i>formativa</i> che <i>riepilogativa</i> : • informare le decisioni di design • identificare problemi di usabilità e trovare soluzioni	Principalmente <i>riepilogativa</i> : • valutare l'usabilità di un sito esistente • monitorare l'usabilità nel tempo • confrontare il sito con i competitor • calcolare il ROI
Quando viene usata	In qualsiasi momento: durante il redesign, o quando si ha un prodotto finale funzionante.	Quando si ha un prodotto funzionante (all'inizio o alla fine di un ciclo di design).
Risultati	Risultati basati sulle impressioni, interpretazioni e conoscenze pregresse del ricercatore.	Risultati statisticamente significativi che probabilmente saranno replicati in uno studio diverso.
Metodologia	• Pochi partecipanti • Condizioni di studio flessibili adattabili alle esigenze del team • Protocollo "think-aloud"	• Molti partecipanti • Condizioni di studio ben definite e rigorosamente controllate • Solitamente senza "think-aloud"

Pianificazione Prima di tutto bisogna definire l'**obiettivo** della ricerca, ovvero che cosa vogliamo testare e a che scopo. Successivamente va deciso il formato, che può essere:

- **In laboratorio o sul campo**
- **Moderato o non moderato:** il primo caso fornisce maggiori informazioni e un miglior accesso ai commenti ma costa di più ed è più difficile da organizzare. Il secondo è più economico, veloce e fornisce un miglior accesso a partecipanti difficili da reclutare
- **Di persona o da remoto:** il primo è il più consigliato quando possibile in quanto fornisce un'interazione più personale che aiuta a capire indizi sottili

Per quanto riguarda il **numero di partecipanti**, Nielsen evidenzia che per studi *qualitativi* 5 persone sono un buon numero. Per quanto riguarda quelli *quantitativi* dovrebbero essere almeno 20. In entrambi i casi devono essere i partecipanti corretti che rispecchiano la demografia e i comportamenti dei *target user*.

Note 3.8.2. Evita di chiedere ai partecipanti di fingere o immaginare scenari in quanto potrebbe invalidare i risultati.

Il passo successivo è quello di scrivere le giuste **task**, che possono essere:

- **Esplorative**, utilizzate per imparare come le persone esplorano le informazioni; non adatte alla ricerca *quantitativa*
- **Specifiche:** sono mirate e hanno una risposta corretta o un punto di arrivo

Infine si decidono le **metriche**, se presenti. Nel caso degli studi *qualitativi* si definiscono le domande da fare ai partecipanti. In quelli *quantitativi* sono invece fondamentali, e.g. tempo speso su una task, livelli di soddisfazione, ratio di successo e di errore.

Note 3.8.3. Può essere molto utile fare prima un **pilot study** per rifinire come sono scritte le task, il loro ordine e quantità e assicurarsi che si stiano reclutando i giusti partecipanti.

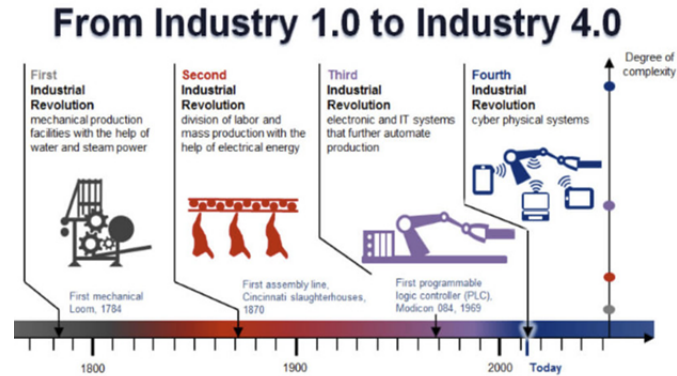
Cosa evitare Ecco dieci cose da evitare durante il testing:

1. Dire agli utenti **dove muoversi** nell'interfaccia
2. Dire agli utenti **cosa fare**
3. Creare task con **date passate**
4. Rendere le task **troppo semplici**
5. Usare scenari **troppo elaborati**
6. Scrivere una **pubblicità** invece di un task
7. Rischiare di causare una **reazione emotiva**
8. Essere **divertenti**
9. **Offendere** i partecipanti
10. **Chiedere come** un partecipante farebbe qualcosa

3.9 IOT

Definizione 3.9.1 (Internet of Things). *Un sistema di computing devices interconnessi, macchine meccaniche e digitali, oggetti, animali o persone che sono dotati di **unique identifiers** (UIDs) e l'abilità di trasferire dati attraverso una rete senza la necessità di interazione umana.*

Definizione 3.9.2 (Industria 4.0). *Descrive l'organizzazione del processo di produzione basato sulla comunicazione automatica di tecnologia e dispositivi lungo la catena produttiva.*



Nell'industria 4.0 i sistemi monitorano i processi finisci e creano una copia virtuale del mondo fisico per poi prendere decisioni decentralizzate basate su meccanismi di auto organizzazione.

Definizione 3.9.3 (Digital Twin). *Una copia digitale di un'entità fisica come processi, persone, luoghi, sistemi e dispositivi.*

3.9.1 Il Ciclo di Vita dell'Esperienza

Il modello analizzato suddivide l'interazione con i sistemi IoT in tre fasi temporali, che superano il semplice momento dell'acquisto:

- **Before Use (Anticipated use)**: la fase di anticipazione e aspettativa verso le potenzialità del dispositivo.
- **During Use (Actual use)**: l'interazione tecnica diretta, dove l'IoT deve garantire efficacia e soddisfazione nel completamento dei task.
- **After Use (Digested use)**: la fase di elaborazione mentale e ritenzione emotiva, cruciale per la fidelizzazione in ecosistemi di servizi ricorsivi.

3.9.2 Livelli di Design e Integrazione Aziendale

La progettazione per l'IoT non si limita all'interfaccia, ma si estende a livelli strategici più profondi:

Usability : Focalizzata sull'efficienza del compito durante l'uso effettivo del dispositivo.

User Experience (UX) : La progettazione dell'esperienza globale legata al singolo prodotto connesso, coprendo l'intero arco temporale (prima, durante e dopo).

Customer Experience (CX) : La gestione di tutti i *touchpoint* dell'ecosistema (es. Advertising, Customer Service, Ethics), vedendo il dispositivo IoT come uno dei molti canali di comunicazione azienda-cliente.

Service Design : Il livello più alto di astrazione che coordina la strategia di business, il personale e il design organizzativo per sostenere l'erogazione del servizio nel tempo.

In sintesi, l'IoT trasforma il prodotto in un'interfaccia di servizio permanente, rendendo il *Service Design* il pilastro fondamentale per la coerenza tra la promessa tecnologica e l'esperienza realmente vissuta dall'utente.

3.9.3 User Experience

Il design per l'IoT è per natura più **complesso** di quello per i servizi web. Questo a causa del livello attuale della tecnologia, dell'immatura comprensione del valore percepito dagli utenti e dal semplice fatto che ci sono più aspetti da considerare, come:

- La natura **specializzata** dei dispositivi IoT: molti hanno bisogno di una forma fisica precisamente progettata e ingegnerizzata e di conseguenza necessitano di più funzionalità di UI
- L'abilità dei dispositivi di connettere il mondo digitale con quello fisico
- Il fatto che molti prodotti sono **sistemi distribuiti** composti da più dispositivi: bisogna capire come distribuire le funzionalità e progettare UI e interazioni su tutto il sistema come un'unica entità, garantendo *coerenza* (**interusability**)
- Il **networking**: nei dispositivi IoT non è sempre detto che ci sia connessione, cosa che invece era assunta nei servizi web
- **Remote configuration**: se prima l'utente aveva una rappresentazione visuale e immediata degli oggetti, con i dispositivi IoT no
- **Power saving**

3.10 Artificial Intelligence

L'intelligenza artificiale è sempre più diffusa in tutti gli ambiti. Il problema principale è che, che funzioni o meno, gli utenti non capiscono come o perché. Richiede quindi degli approcci fondamentalmente diversi di User Experience.

3.10.1 Differenze

A differenza del software tradizionale, la IA è **probabilistica**, fornisce **falsi positivi** e **negativi**, si basa su un principio **black-box** dove il processo decisionale è nascosto e ha **comportamenti imprevedibili** che possono essere confusionari, pericolosi o distruttivi.

Definizione 3.10.1 (Human-Centered AI). *AI che amplifica, aumenta e migliora le performance umane mantenendo i sistemi affidabili e sicuri.*

Questo approccio vuole quindi far **collaborare** umani e IA in maniera trasparente e con livelli di controllo appropriati. Inoltre è fondamentale lasciare alcune sezioni dell'interfaccia *deterministiche* in modo che, anche quando l'IA non è sicura (che dovrebbe essere il meno possibile), gli utenti possono ricadere su interazioni prevedibili.

3.10.2 Affidamento

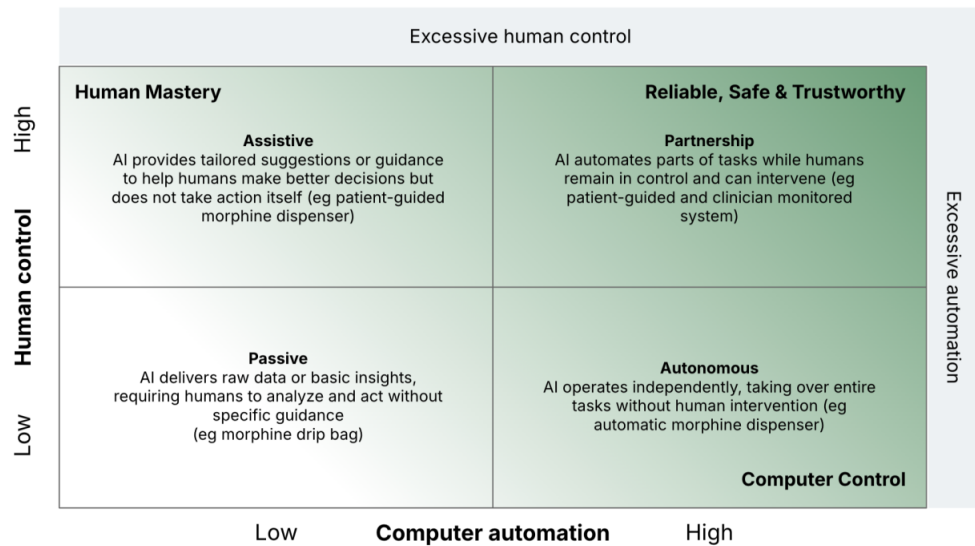
Il problema dell'affidamento è che gli utenti potrebbero usare l'IA **troppo** o **ignorarla completamente** anche se potrebbe essere utile. È fondamentale trovare la giusta via di mezzo basata anche sul livello di *accuratezza* e sull'importanza delle conseguenze.

A differenza della **fiducia**, che è psicologica e difficile da misurare, l'*affidamento* è osservabile, quantificabile e può essere calibrato in base alle performance.

Per costruire un affidamento appropriato bisogna fornire questi tre meccanismi:

- **Explainability**: aiutare gli utenti a capire il perché mostrando ciò che un sistema può fare e come gli input influenzano i risultati, rimanendo specifici sui livelli di performance
- **Trasparenza**: aiutare gli utenti a capire quando affidarsi, mostrando le performance per categoria, i livelli di confidence delle predizioni e avvisando quando la situazione è diversa dai dati di training
- **Controllo**: gli utenti devono poter controllare i dati di input, le informazioni personali e i risultati

È importante capire che **augmentation** (AI per suggerire alternative ed eseguirle se approvate) e **automation** (AI prima esegue e poi informa quando vuole) esistono su uno **spettro**. Bisogna capire a che livello dare controllo agli umani.



3.10.3 Miglioramento

L'IA impara ed evolve nel tempo consumando dati e ricevendo **feedback** sugli output, che possono essere di tre tipi feedback:

- **Esplicito**: l'utente lo dice esplicitamente per migliorare il sistema, e.g. like/dislike
- **Implicito**: dedotto dal comportamento dell'utente e le sue azioni, e.g. skip rate
- **Duale**: combinazione dei due

È importante fare la scelta in base al contesto, impegno e qualità del segnale necessari.

Nella pratica il feedback deve essere semplice da dare, mostrare subito l'impatto che ha avuto e rispettare il contesto, quindi non essere utilizzato troppo.

3.10.4 Design

Vediamo tre principi fondamentali:

- **Aspettative chiare**: mostrare cosa il sistema può e non può fare, con quale accuratezza e limitazioni e come fa ad imparare
Note 3.10.1. Sempre meglio dare basse aspettative e poi fare più del previsto, non il contrario.
- **Design per gli errori**: esistono diversi tipi di errore
 - **System**: problemi tecnici e crash
 - **Model**: predizioni sbagliate e allucinazioni
 - **Data**: dati di training di bassa qualità o input prevenuti
 - **Relevance**: risultato corretto ma irrilevante
 - **User**: input non chiaro, errore di comprensione

Per gestirli bisogna indicare quando si presentano, spiegare cosa è andato storto e offrire opportunità per correggere. Bisogna sempre permettere di tornare indietro e dare il controllo all'utente.

- **Bilanciare complessità e controllo**: mostrare prima le funzionalità base e aggiungere le altre nel tempo, sempre rispettando l'autonomia dell'utente e la possibilità di scartare le decisioni dell'IA

I requisiti di design cambiano molto anche in base a quanto c'è in gioco:

- **High-stakes:** serve massima trasparenza, controllo ed explainability in quanto un errore potrebbe avere conseguenze catastrofiche (e.g. medicina)
- **Medium-stakes:** serve una buona trasparenza e moderato controllo, e.g. educazione
- **Low-stakes:** serve un livello base di trasparenza, e.g. autocompletamento

	Positivo Predetto	Negativo Predetto
Positivo Reale	Vero Positivo (VP) Predizione corretta	Falso Negativo (FN) Rilevamento mancato
Negativo Reale	Falso Positivo (FP) Falso allarme	Vero Negativo (VN) Rifiuto corretto

Precision e Recall

Definizione 3.10.2 (Precisione). *Quando ha trovato qualcosa, quanto spesso è corretto?*

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Definizione 3.10.3 (Recall). *Di tutte le cose che dovrebbe trovare, quante ne ha trovate effettivamente?*

$$\frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

È importante ottimizzare uno dei due parametri a seconda del **contesto**: la *precisione* serve quando la cosa peggiore è un falso allarme (e.g. spam detection) mentre la *recall* quando è un positivo mancante (e.g. analisi mediche).

Processo Il processo di design per la IA itera su tre cose contemporaneamente: UI, modello di IA e dati. Di seguito i passaggi:

1. **Comprendere** le **necessità** dell'utente, prestando attenzione al *livello di controllo*, quanto c'è in gioco e come l'utente costruirà la fiducia nel tempo
2. **Prototype** anticipato con un test di tipo *Wizard of Oz* per valutare i pattern di interazione
3. **Definire i requisiti** del modello collaborando con data scientists e misurando varie metriche tra cui anche le performance umane e la soddisfazione degli utenti
4. **Test** con dati veri: la qualità dei dati è fondamentale e bisogna trovare strategie per ottenerli
5. **Collaborazione** durante tutto il processo tra gli sviluppatori e i designer